

Munasabat

Occasions

Wedding Planner & Guest Management Platform

Mobile Application Development — Project Report

Platform: Flutter
Backend: Firebase
Payments: Stripe
Maps: OpenStreetMap
State Mgmt: Riverpod
Navigation: go_router

Submitted By:
Abdullah Al Mazid Zomader
ID: 24241189

August 23, 2026

Abstract

Munasabat is a cross-platform mobile application built with Flutter that consolidates the entire wedding planning lifecycle into a single, unified dashboard. The application addresses a clear gap in the wedding-planning market: today's wedding hosts rely on a fragmented combination of spreadsheets, WhatsApp groups, paper guest lists, and printed invitations. Munasabat replaces all of these with a structured, digital, and collaborative platform.

The application serves two contextual roles within a single user account: a **Host** who creates and manages events, and a **Guest** who receives invitations and participates. Key features include event creation with interactive map-based venue selection, a checklist task manager, a budget tracker, digital invitation generation with QR codes, a join-request and manual check-in system, digital cash gifting via the Stripe sandbox, a shared wedding memory gallery, and in-app purchase of premium features.

This report documents the complete technical design, architecture, data model, development phases, implementation decisions, and testing approach for the Munasabat coursework build. All payment flows are executed against Stripe's **sandbox/test environment**; no real funds are captured or moved at any point.

Contents

1	Introduction	2	6.2	Route Map	7
1.1	Background and Motivation	2	6.3	Sidebar Drawer Navigation	7
1.2	Project Scope	2	7	Stripe Sandbox Payment Integration	7
1.3	Report Structure	2	7.1	Architecture Decision: Client-Side PaymentIntent Creation	7
2	Technology Stack	2	7.2	Payment Flow	7
2.1	Overview	2	7.3	Shared Integration	7
2.2	Why Flutter?	2	7.4	Configuration	7
2.3	Why Firebase?	2	8	Image Storage Strategy	7
2.4	Why Stripe over Other Gateways?	2	8.1	Why Base64 in Firestore?	7
2.5	Why OpenStreetMap over Google Maps?	2	8.2	Compression Pipeline	7
3	Data Model	3	8.3	Rendering	7
3.1	Firestore Collections Overview	3	9	Design System	8
3.2	Collection Hierarchy Summary	4	9.1	Brand Personality	8
4	Application Architecture	4	9.2	Color Palette	8
4.1	Layered Architecture	4	9.3	Typography	8
4.2	Folder Structure	4	9.4	Spacing & Layout	8
4.3	State Management with Riverpod	4	9.5	Component Library	8
5	Development Phases and Features	4	10	Security and Data Privacy	8
5.1	Phase 0 — Project Setup & Foundations	4	10.1	Firestore Authentication	8
5.2	Phase 1 — Authentication	4	10.2	Firestore Security Rules	8
5.3	Phase 2 — Unified Dashboard & Event Creation	5	10.3	Payment Security	8
5.4	Phase 3 — Checklist & Budget Tracker	5	10.4	Privacy	8
5.5	Phase 4 — Guest Management	5	11	Key Architectural Decisions	8
5.6	Phase 5 — Event Sharing & Digital Invitations	5	12	Testing and Verification	9
5.7	Phase 6 — Attending Events & Join Flow	6	12.1	Approach	9
5.8	Phase 7 — Manual Check-In	6	12.2	Phase-by-Phase Verification Checklist	9
5.9	Phase 8 — Wedding Gifts / Cash Gifts	6	12.3	Error Handling Verification	9
5.10	Phase 10 — Wedding Memories Gallery	6	12.4	Stripe Sandbox Testing	9
5.11	Phase 11 — Premium Features	6	13	Conclusion	9
5.12	Phase 12 — Gift Withdrawals	6	14	AI Usage Statement	9
6	Navigation	7			
6.1	go_router Configuration	7			

1 Introduction

1.1 Background and Motivation

Wedding planning is one of the most logistically demanding personal events a person organizes. Hosts must simultaneously track dozens of tasks (booking venues, caterers, photographers), manage a growing guest list, communicate event details to hundreds of people, and track a budget spanning multiple categories. Guests need to receive formal invitations, RSVP, locate the venue, and increasingly wish to send digital gifts.

Today this workflow is fragmented: hosts maintain guest lists in Excel spreadsheets, share event details via WhatsApp messages, and receive RSVPs through phone calls. There is no single platform tailored to this need in the South Asian and Middle Eastern markets where weddings are often large, multi-day affairs.

Munasabat was conceived to solve exactly this problem. By consolidating event management, guest communication, digital gifting, and shared memories into one mobile application, it gives hosts a command center and guests a seamless participation portal.

1.2 Project Scope

This build is an academic **Minimum Viable Product (MVP)** targeting the following core experiences:

- **Unified authentication:** a single email-based login system for all users.
- **Event lifecycle management:** create, edit, and delete wedding events with rich metadata.
- **Planning tools:** a per-event checklist manager and budget tracker.
- **Guest management:** manual guest entry, join-request approval, and manual check-in.
- **Digital invitations:** a shareable invitation card with a unique Event Code and QR code.
- **Cash gifting:** guests send monetary gifts via the Stripe sandbox payment sheet.
- **Wedding memories:** a shared photo gallery stored using Base64 encoding in Firestore.
- **Premium features:** hosts unlock expanded guest and photo limits via sandbox purchases.

1.2.1 Explicitly Out-of-Scope Items

- Real (live) payment processing — all Stripe flows are sandbox only.
- Server-side Cloud Functions (Stripe keys are managed client-side for MVP).
- Personal per-guest QR check-in scanner (manual check-in replaces it).
- Multi-language support.
- Seating chart or table assignment tools.
- Web platform (Flutter mobile only).
- Platform-level admin or moderation dashboard.
- Contact import from the device address book.
- Wedding vendor directory.

1.3 Report Structure

This report details the end-to-end engineering of Munasabat across key architectural layers: Section 2 covers the technology stack; Section 3 details Firestore data models; Section 4 outlines the application architecture and Riverpod state management; Section 5 chronicles development phases; Section 6 explains navigation; Section 7 covers Stripe payment workflows; Section 8 details image compression and storage; Section 9 outlines the design system; Section 10 discusses security rules; Section 11 provides architectural decisions; Section 12 covers testing and verification; Section 13 presents conclusions; and Section 14 provides the AI usage statement.

2 Technology Stack

2.1 Overview

Munasabat is built entirely on the **Flutter** framework with **Firestore** as its backend. Every library and service was chosen to satisfy two constraints: (1) stay within free-tier infrastructure limits suitable for an academic project, and (2) use well-maintained, community-standard packages appropriate for a production-quality codebase.

2.2 Why Flutter?

Flutter was selected for its ability to compile a single Dart codebase into native iOS and Android applications. For an academic MVP targeting two platforms, this eliminates the need to maintain two separate codebases (Swift/Kotlin). Flutter's widget system also enabled the rich, branded UI described in Section 9 to be implemented consistently across both platforms.

2.3 Why Firebase?

Firebase's Spark (free) tier provides:

- **Firestore Authentication:** unlimited email/password users at no cost.
- **Cloud Firestore:** 1 GB storage, 50,000 daily reads, 20,000 daily writes — sufficient for this coursework scale.
- **Real-time listeners:** Firestore's `snapshots()` streams eliminate the need for polling, enabling an instantaneous UI (e.g., join requests appear the moment a guest submits one).

2.4 Why Stripe over Other Gateways?

SSLCOMMERZ, an alternative gateway considered for this project, requires a server-side integration to generate payment sessions, which would have necessitated setting up a Node.js backend or Firebase Cloud Functions. Stripe's `flutter_stripe` package supports a client-side `PaymentIntent` creation flow using the Secret Test Key, allowing the MVP to process sandbox payments without any server.

2.5 Why OpenStreetMap over Google Maps?

Google Maps Platform requires a billing account even for the free tier. OpenStreetMap via `flutter_map` is completely free, with Nominatim providing a free geocoding API for venue name search. The resulting interactive map experience is functionally equivalent for the venue-selection use case.

Table 1: Technology Stack Summary

Layer	Technology	Package / Tool	Notes
Framework	Flutter (stable channel)	SDK	Single codebase for iOS + Android
Language	Dart (null-safety enabled)	SDK	Strong typing, async/await throughout
State Mgmt	Riverpod	flutter_riverpod	AsyncNotifier/StateNotifier pattern
Navigation	Declarative routing	go_router	Declarative routes with auth guards
Auth	Firebase Authentication	firebase_auth	Email/password; session persistence
Database	Cloud Firestore	cloud_firestore	NoSQL, real-time listeners
File Storage	Base64 strings in Firestore	Internal pipeline	Avoids Firebase Storage billing
Payments	Stripe (sandbox only)	flutter_stripe	Test mode payment sheet
Maps	OpenStreetMap	flutter_map + OSM	Free alternative to Google Maps
QR Codes	QR Generation	qr_flutter	Event QR code generation
Image Picking	Image Picker	image_picker	Gallery + camera support
Sharing	Native Share Sheet	share_plus	Native OS share sheet
Date / Currency	Formatting	intl	Locale-aware dates and currency
HTTP Client	HTTP	http	Direct Stripe API calls (client-side)

3 Data Model

3.1 Firestore Collections Overview

Munasabat uses a NoSQL document-oriented schema in Cloud Firestore. Top-level documents and subcollections are organized to reflect the ownership hierarchy: events own checklists, expenses, guests, memories, gifts, and withdrawals.

3.1.1 users/{userId}

```
{
  "uid": "string",
  "email": "string",
  "displayName": "string",
  "avatarBase64": "string | null",
  "phone": "string | null",
  "createdAt": "Timestamp"
}
```

3.1.2 events/{eventId}

```
{
  "id": "string",
  "hostId": "string",
  "title": "string",
  "description": "string",
  "eventDate": "Timestamp",
  "venueName": "string",
  "latitude": "double",
  "longitude": "double",
  "coverImageBase64": "string | null",
  "eventCode": "string (6 chars)",
  "totalBudget": "double",
  "createdAt": "Timestamp"
}
```

3.1.3 events/{id}/checklist/{itemId}

```
{
  "id": "string",
  "title": "string",
  "category": "string",
  "isCompleted": "boolean",
  "dueDate": "Timestamp | null",
  "createdAt": "Timestamp"
}
```

3.1.4 events/{id}/expenses/{expenseId}

```
{
  "id": "string",
  "title": "string",
  "category": "string",
  "amount": "double",
  "paidBy": "string",
  "createdAt": "Timestamp"
}
```

3.1.5 events/{id}/guests/{guestId}

```
{
  "id": "string",
  "name": "string",
  "email": "string | null",
  "phone": "string | null",
  "status": "pending | accepted | declined | checkedIn",
  "isAttending": "boolean",
  "invitedVia": "manual | link | code",
  "userId": "string | null",
  "createdAt": "Timestamp"
}
```

3.1.6 events/{id}/memories/{memoryId}

```
{
  "id": "string",
  "imageBase64": "string",
  "caption": "string",
  "uploadedBy": "string (userId)",
  "uploadedByName": "string",
  "createdAt": "Timestamp"
}
```

3.1.7 events/{id}/gifts/{giftId}

```
{
  "senderId": "string (userId)",
  "displayName": "string | null (null if anon)",
  "amount": "integer (cents)",
  "currency": "string (e.g. \"usd\")",
  "message": "string | null",
  "isAnonymous": "boolean",
  "paymentStatus": "\"succeeded\"",
  "stripePaymentIntentId": "string",
  "createdAt": "Timestamp"
}
```

The `senderId` is always stored regardless of anonymity so the data remains auditable.

3.1.8 events/{id}/withdrawals/{withdrawalId}

```
{
  "amount": "integer (cents -- 90% of total gifts)",
  "status": "\"completed\"",
  "createdAt": "Timestamp"
}
```

3.1.9 events/{id}/premium_status/status

A single document (not a collection) storing the premium feature state for an event.

```
{
  "isPremium": "boolean",
  "imageLimit": "integer (default: 5)",
  "unlockedFeatures": ["string"],
  "stripePaymentIntentId": "string | null"
}
```

3.2 Collection Hierarchy Summary

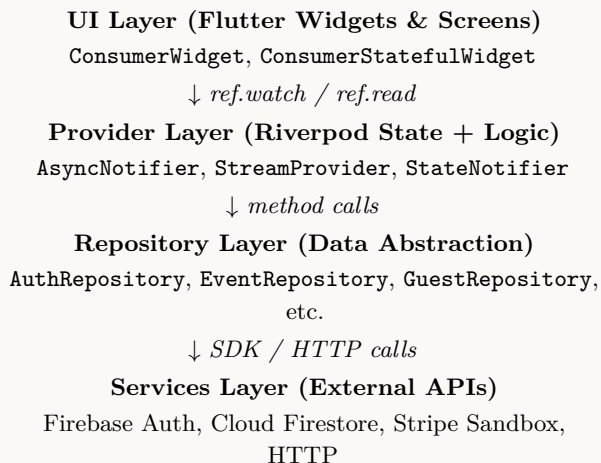
```
users/{userId}
events/{eventId}
  |-- checklist/{itemId}
  |-- expenses/{expenseId}
  |-- guests/{guestId}
  |-- memories/{memoryId}
  |-- gifts/{giftId}
  |-- withdrawals/{withdrawalId}
  '-- premium_status/status
```

4 Application Architecture

4.1 Layered Architecture

Munasabat follows a strict **four-layer architecture** that separates concerns between UI, state management, data access, and external services:

Four-Layer System Architecture



This architecture enforces the following invariants:

- **Widgets never call Firebase or Stripe directly.** They only read and watch Riverpod providers.
- **Providers never contain raw Firestore query strings.** All query logic lives in repositories.
- **Repositories are the only layer** that imports `cloud_firestore` and `firebase_auth`.
- **Business logic never lives in widget build() methods.** Only simple conditional rendering is acceptable in the UI layer.

4.2 Folder Structure

The project follows a **feature-first** folder structure under `lib/`:

```
lib/
|-- main.dart
|-- app.dart # MaterialApp + GoRouter
|-- core/
| |-- routing/ # go_router config
| |-- services/ # stripe_service.dart
| |-- theme/ # AppColors, AppTextStyles
| '-- widgets/ # Reusable UI widgets
|-- models/ # Data classes with JSON serialization
|-- repositories/ # Data access abstraction
|-- providers/ # Riverpod state controllers
'-- features/ # Screen UI by domain
```

An important architectural rule: **one Dart file per feature module**. All screens and sub-widgets for a feature are consolidated into a single, cohesive `.dart` file under that feature's folder, making each feature self-contained.

4.3 State Management with Riverpod

Riverpod is the sole state management solution. The project uses the provider types summarized in Table 2. All loading/error/data states are represented through Riverpod's `AsyncValue` type, surfaced to the UI using the `when(data:, loading:, error:)` pattern. This ensures every async operation has a consistent three-state UI representation.

5 Development Phases and Features

The project was built in sequential, demoable phases. Each phase had a defined goal and exit criteria that had to be met before the next phase began.

5.1 Phase 0 — Project Setup & Foundations

Goal: A running Flutter app connected to Firebase with theming and navigation skeleton in place.

What was built:

- Initialized Flutter project and configured the folder structure per the architecture document.
- Added all approved dependencies to `pubspec.yaml`.
- Linked the app to Firebase project `munasabat-2d9c2` via `flutterfire configure`.
- Created all Dart models in `lib/models/` for every Firestore collection.
- Initialized Firebase in `lib/main.dart`.
- Established the project theme, font bundles (Playfair Display, Poppins), and `ThemeData`.

5.2 Phase 1 — Authentication

Goal: Full auth flow (register, login, logout, password reset) for email/password.

What was built:

- `AuthRepository` encapsulating all Firebase Auth operations (register, login, logout, password reset, avatar update).
- `AuthProvider` (Riverpod `AsyncNotifier`) for session persistence and state.

Table 2: Riverpod Provider Types Used in Munasabat

Provider Type	Usage	Example
<code>StreamProvider</code>	Real-time Firestore listeners	<code>hostEventsProvider</code>
<code>StreamProvider.family</code>	Parameterized stream queries	<code>expensesProvider(eventId)</code>
<code>FutureProvider</code>	One-shot async reads	<code>currentUserModelProvider</code>
<code>StateNotifierProvider</code>	UI state + operations controller	<code>expenseControllerProvider</code>
<code>Provider</code>	Singleton service instances	<code>giftRepositoryProvider</code>
<code>AsyncNotifier</code>	Auth session management	<code>authNotifierProvider</code>

- **LoginScreen** with email/password fields and a “Forgot Password” bottom sheet.
- **RegisterScreen** for new user signup (Name, Email, Password, optional Phone).
- **AppRouter** (`GoRouter`) with auth-state-based route guards.
- Initial Firestore Security Rules.

Key decision: Phone/OTP login was removed by product owner decision. Phone is collected only as an optional profile field. Email is the sole authentication method.

Error handling: `FirebaseAuthException` codes are mapped to plain-language messages. For example, `email-already-in-use` maps to “An account with that email already exists. Try logging in instead.”

5.3 Phase 2 — Unified Dashboard & Event Creation

Goal: Users can create events and see a unified overview of events they host and attend.

What was built:

- An animated splash screen (logo zoom-in with `Curves.elasticOut`, 1.2s) that auto-navigates to Login.
- **Unified Dashboard** with “Hosting” and “Attending” tabs, and a sidebar drawer navigation.
- **Stats Grid** (2×2): live counts of hosted events, attending events, checklist completion rate, and total budget spent, powered by real Firestore stream providers.
- **Event Management:** full CRUD for events with a rich create/edit form.
- **Interactive Map Picker:** `flutter_map` with Open-StreetMap tiles, Nominatim geocoding search for venue lookup, and the selected coordinates saved to Firestore.
- **Event Detail Screen:** shows all event metadata with a mini-map view and an “Open in Maps” deep link.

Key decision: Bottom navigation bar replaced with a sidebar drawer as the primary navigation pattern, per product owner preference.

5.4 Phase 3 — Checklist & Budget Tracker

Goal: Host can manage per-event planning tasks and expenses.

Checklist Manager:

- `ChecklistRepository` and `ChecklistProvider` for per-event task management.

- Add task, toggle completion with a strike-through animation, swipe-to-delete.
- A progress bar showing the percentage of completed tasks.

Budget Tracker:

- `totalBudget` field added to `EventModel` (stored in cents).
- `ExpenseRepository` and `ExpenseController` (Riverpod `StateNotifier`).
- Add expense via a bottom sheet (label, category, amount).
- Summary card showing Total Budget, Total Spent, and Remaining Balance with a visual progress bar.
- Category-based expense list with swipe-to-delete.

The stats grid aggregates checklist completion and budget data across all hosted events via a `dashboardStatsProvider`.

5.5 Phase 4 — Guest Management

Goal: Host can build and manage a guest list and process incoming join requests.

What was built:

- Manual guest entry (Name, Phone, Email) with form validation.
- Real-time searchable guest list.
- Swipe-to-delete and edit-guest functionality.
- **Join Requests section:** guests who have submitted a join request (`status: "requested"`) appear in a visually distinct block at the top of the list.
- Host can instantly **Approve** (status changes to `accepted`) or **Reject** (guest document deleted) requests in real time.

Free-tier cap: The guest list enforces a **20-guest limit** on the free tier. A locked FAB and upgrade banner appear once the cap is reached.

5.6 Phase 5 — Event Sharing & Digital Invitations

Goal: Each event has a unique Event Code and QR code for sharing.

What was built:

- On event creation, a unique 6-character `eventCode` (e.g., `MNSB-XK9F3A`) is generated and stored in the event document.

- A `qrToken` (UUID) is also generated for future QR-scan support.
- **InvitationScreen**: a beautiful photocard showing event title, host name, date/time, venue, and a dynamically generated QR code via `qr_flutter`.
- Native share sheet integration (`share_plus`) for sharing via WhatsApp, SMS, or clipboard.

5.7 Phase 6 — Attending Events & Join Flow

Goal: Guests can request to join an event using the Event Code.

What was built:

- “Join Event” floating action button on the dashboard.
- **JoinEventModal**: user enters a 6-character Event Code; the app looks up the event and creates a guest document with `status: "requested"`.
- The event appears in the “Attending” tab with a “Pending Approval” label until the host approves.
- **AttendingScreen**: full event detail view for approved guests (date, venue, map link).

5.8 Phase 7 — Manual Check-In

Goal: Host can manually check guests in at the event.

What was built:

- A “Check In” button appears on guest list items for guests with `status: "accepted"`.
- Tapping it updates the status to `"checked_in"` and records a `checkedInAt` timestamp.
- The UI updates instantly via the Firestore real-time listener.

5.9 Phase 8 — Wedding Gifts / Cash Gifts

Goal: Guests can send a digital cash gift to the host through the Stripe sandbox.

Guest-side flow:

1. Guest opens the “Send Gift” screen for an event.
2. Enters an amount, an optional personal message, and toggles “Send Anonymously”.
3. The app calls `StripeService` to generate a `PaymentIntent` via a direct HTTP POST to api.stripe.com.
4. Stripe SDK launches the native Payment Sheet UI (card entry).
5. On success, the client validates the intent status with an HTTP GET to Stripe.
6. A `GiftModel` document is written to `events/{eventId}/gifts/` in Firestore.

Host-side (Gift Wallet):

- Total received, gift count, platform commission (10%), and available balance (90%).
- Real-time list of all gifts with sender name (or “Anonymous Well-Wisher”), amount, message, and timestamp.

- Withdrawal flow: host taps “Withdraw” and the available balance is cleared with a withdrawal record saved to Firestore.

Guest-side history: Guest can view all gifts they have sent via a Firestore `collectionGroup` query on the `gifts` collection filtered by `senderId`.

5.10 Phase 10 — Wedding Memories Gallery

Goal: Post-event shared photo gallery for hosts and approved guests.

What was built:

- Host and approved guests can upload photos via `image_picker` (Camera or Gallery).
- Images are compressed locally to max 800×800 px at 50% JPEG quality before upload.
- Compressed bytes are Base64-encoded and stored in Firestore.
- **MemoriesScreen**: a real-time `GridView` of all uploaded photos.
- Full-screen image viewer with pinch-to-zoom and swipe navigation.
- Host-only delete: hosts see a delete icon on each photo tile.
- **5-photo free limit**: a progress counter strip below the AppBar shows photos used vs. the limit.

5.11 Phase 11 — Premium Features

Goal: Hosts can purchase premium features for an event via a sandbox payment.

Available premium products:

- **Unlimited Guests** (\$9.99): removes the 20-guest free-tier cap.
- **+100 Image Pack** (\$9.99): increments the `imageLimit` field in `premium_status` by 100.

Both products use the same Stripe sandbox integration as the gifting feature. On confirmed success, the `premium_status` document is updated and the relevant cap is lifted immediately via the real-time Firestore listener — no app restart required.

5.12 Phase 12 — Gift Withdrawals

Goal: Hosts can withdraw their available gift balance.

What was built:

- Withdrawal UI in the Gift Wallet screen showing the Available Balance (90% of total received).
- Tapping “Withdraw” creates a withdrawal record in the `withdrawals` subcollection.
- The available balance is recalculated by subtracting all past withdrawals from the 90% share.
- This is a simulated payout — no real funds are moved.

6 Navigation

6.1 go_router Configuration

All navigation is handled by `go_router` using declarative routes. The router is configured with a redirect guard that checks the auth state:

- **Unauthenticated users** are always redirected to `/login`.
- **Authenticated users** landing on `/login` or `/` are redirected to `/dashboard`.

6.2 Route Map

Table 3: Application Routes

Route	Screen Description
<code>/</code>	Animated Splash Screen
<code>/login</code>	Login / Register / Forgot PW
<code>/dashboard</code>	Unified Dashboard (Tabs)
<code>/event/create</code>	Create Event Form
<code>/event/:eventId</code>	Event Detail View
<code>/event/:eventId/edit</code>	Edit Event Form
<code>/event/:eventId/guests</code>	Guest List & Requests
<code>/event/:eventId/checklist</code>	Checklist Manager
<code>/event/:eventId/budget</code>	Budget Tracker
<code>/event/:eventId/invitations</code>	Digital Invitation Card
<code>/event/:eventId/memories</code>	Wedding Photo Gallery
<code>/event/:eventId/gifts</code>	Gift Wallet (Host view)
<code>/event/:eventId/send-gift</code>	Send Cash Gift (Guest)
<code>/event/:eventId/premium</code>	Premium Upgrade Paywall
<code>/attending/:eventId</code>	Guest Event Detail View

6.3 Sidebar Drawer Navigation

The primary navigation mechanism is a **sidebar drawer** accessible from the dashboard. The drawer provides direct links to: My Events (hosted), My Invitations (attending), event-specific sub-links (Guests, Budget, Checklist, Memories, Gifts, Invitations), User Profile, and Logout. This replaced a bottom navigation bar in favor of the drawer's richer contextual hierarchy.

7 Stripe Sandbox Payment Integration

7.1 Architecture Decision: Client-Side PaymentIntent Creation

In a production application, generating a Stripe `PaymentIntent` requires a server-side endpoint to protect the Secret Key. For this academic MVP, the `PaymentIntent` is generated directly from the Flutter client via HTTP POST to https://api.stripe.com/v1/payment_intents using the Stripe Test Secret Key. This approach:

- Bypasses the need for any backend server or Cloud Functions.
- Is **explicitly permitted** for this coursework build.
- Would be replaced by a server-side Cloud Function in a production deployment.

Security Note

The Stripe Secret Key must never be included in a production app binary. In production, a server-side endpoint would accept payment parameters, generate the `PaymentIntent`, and return only the client secret to the app. The Secret Key stays exclusively on the server.

7.2 Payment Flow

1. Client calls `StripeService.createPaymentIntent` via HTTP POST to Stripe's API endpoint.
2. The returned `client_secret` is used to initialize the Stripe Payment Sheet.
3. `Stripe.instance.presentPaymentSheet()` launches the native payment UI.
4. On success, the client calls `checkPaymentIntentStatus` to confirm the payment actually succeeded (handling Android SDK false-negative exceptions).
5. A gift or premium record is written to Firestore with the confirmed `stripePaymentIntentId`.

7.3 Shared Integration

Both the **Wedding Gifts** and **Premium Features** flows share a single `StripeService` implementation. The only difference is the Firestore document written on success.

7.4 Configuration

- Payment methods are restricted to **card only** (Stripe Link disabled) to keep the sandbox experience simple.
- Redirect URL `munasabat://stripe-redirect` is configured for 3DS authentication redirects.
- `AndroidManifest.xml` includes a deep-link `intent-filter` for the `munasabat://` scheme.
- Stripe test card: 4242 4242 4242 4242, any future expiry, any CVC.
- Money is stored as integers in cents (e.g., \$9.99 = 999 cents) to avoid floating-point errors.

8 Image Storage Strategy

8.1 Why Base64 in Firestore?

Firebase Storage (for binary file hosting) requires the Blaze (pay-as-you-go) billing plan. To keep this project strictly within the free Spark tier, all images (avatars, event cover photos, gallery memories) are stored as **Base64-encoded strings directly in Firestore documents**.

8.2 Compression Pipeline

Before any image is stored, the client applies local compression:

Table 4: Image Compression Settings

Image Type	Max Dims	Quality
User avatar	400×400 px	50% JPEG
Event cover photo	800×800 px	50% JPEG
Memory / gallery	800×800 px	50% JPEG

This ensures that each image fits within Firestore's 1 MB document size limit after Base64 encoding (which adds approximately 33% overhead to the raw byte size).

8.3 Rendering

When displaying stored images, the app decodes the Base64 string using Dart's `base64Decode()` and renders it with `Image.memory()`. The rendering code checks the header bytes to identify the format (JPEG starts with `/9j/` in Base64; PNG starts with `iVBORw`). If neither prefix matches, the app falls back to a placeholder image.

9 Design System

9.1 Brand Personality

Munasabat’s visual identity is rooted in the project’s meaning: *Occasions*. The design language is **elegant, warm, and celebratory** — reminiscent of a premium wedding invitation. Color is used sparingly, like watercolor washes: soft and diffuse, concentrated at moments of delight (hero headers, empty states, invitations), never as flat fills on functional screens.

9.2 Color Palette

All colors are defined in a centralized `AppColors` class under `core/theme/`, and referenced via `Theme.of(context)` throughout the app. No hex values are hardcoded in widget files.

Table 5: Brand Color Tokens

Token	Hex	Usage
<code>brandInk</code>	<code>#371E08</code>	Wordmark, primary CTAs
<code>brandInkLight</code>	<code>#6B4A2E</code>	Pressed state, secondary
<code>accentBlue</code>	<code>#7BAAFA</code>	Links, info highlights
<code>accentBlueSoft</code>	<code>#A4E6F6</code>	Chip fills, card bgs
<code>accentPink</code>	<code>#F99CEC</code>	Celebratory highlights
<code>accentPinkSoft</code>	<code>#EAC0FF</code>	Soft pink fills
<code>ink</code>	<code>#2E2318</code>	Primary body text
<code>charcoal</code>	<code>#5C5248</code>	Secondary text
<code>stone</code>	<code>#9B9188</code>	Tertiary text, disabled
<code>cream</code>	<code>#FBF9F6</code>	App background
<code>surface</code>	<code>#FFFFFF</code>	Cards, sheets
<code>divider</code>	<code>#EFEAE3</code>	Borders, dividers

Table 6: Semantic Status Color Tokens

Token	Hex	Status Meaning
<code>statusPending</code>	<code>#B8860B</code>	Invitation Pending
<code>statusAccepted</code>	<code>#2E7D4F</code>	Accepted / Success
<code>statusDeclined</code>	<code>#B23A3A</code>	Declined / Error
<code>statusCheckedIn</code>	<code>#2E5E8C</code>	Checked In at venue

9.3 Typography

Two typefaces are bundled in the app (self-hosted, not loaded at runtime):

- **Playfair Display** (serif): celebratory/wedding-facing headings.
- **Poppins** (sans-serif): functional UI — forms, lists, buttons, admin screens.

9.4 Spacing & Layout

- **Base unit:** 4px. All spacing uses multiples: 4, 8, 12, 16, 24, 32, 48.
- **Screen horizontal padding:** 16px default.
- **Card corner radius:** 16px.
- **Button corner radius:** 12px.
- **Input field corner radius:** 10px.
- **Minimum tap target:** 48 × 48 px for all interactive elements.

9.5 Component Library

Key shared components defined in `core/widgets/`:

PrimaryButton: Filled `brandInk` background, white text, 12px radius.

AppTextField: Consistent input field with 10px radius, `divider` border, focus highlight.

LoadingOverlay: Full-screen semi-transparent overlay with centered circular progress indicator.

StatusChip: Pill-shaped badge colored by the status token. Always includes text (color alone is never used to convey meaning).

ErrorSnackBar / SuccessSnackBar: Standardized snack bars with plain-language error text.

10 Security and Data Privacy

10.1 Firebase Authentication

- Passwords are never stored or handled by the app — they are managed entirely by Firebase Auth’s secure flow.
- Session persistence is handled by Firebase Auth’s built-in token refresh. The `authStateChanges` stream drives the router’s auth guard.
- After registration, the user is immediately signed out so they must log in explicitly (preventing auto-login after signup).

10.2 Firestore Security Rules

Security rules enforce that:

- A user can only read/write their own `users/{userId}` document.
- Event CRUD is restricted to the event’s `hostId` field matching the authenticated user’s UID.
- Guest subcollection writes are restricted: only the event host can add/edit/delete guests (except creating a join-request document when a guest submits a code).
- Memory galleries are private: only approved guests (or the host) of an event can read the `memories` subcollection.
- `gifts` subcollection: guests can write (send a gift); only the host can read all gifts for their event.

10.3 Payment Security

- All payment flows use Stripe’s sandbox API keys. There is no code path in this build that can charge a real card.
- Anonymous gift anonymity is enforced at the UI level. The `displayName` field is `null` for anonymous gifts, but the underlying `senderId` is always stored for auditability.
- Money is always stored as integers in the smallest currency unit (cents) to avoid floating-point errors.

10.4 Privacy

- No analytics or third-party tracking SDKs are used.
- No real personal data is hardcoded in sample or seed data.
- Private galleries cannot be accessed by unauthenticated users or non-invited guests, enforced at the Firestore Security Rules level.

11 Key Architectural Decisions

Throughout the engineering lifecycle of Munasabat, key technical and architectural decisions were established to

Table 7: Typography Scale

Style	Font Family	Size	Weight	Usage Context
displayLarge	Playfair Display	32pt	700	Invitation title, event hero
headlineMedium	Playfair Display	24pt	600	Section headers
titleLarge	Poppins	20pt	600	Screen titles
titleMedium	Poppins	16pt	600	Card titles, list items
bodyLarge	Poppins	16pt	400	Primary body text
bodyMedium	Poppins	14pt	400	Secondary body, descriptions
labelSmall	Poppins	12pt	500	Badges, status chips, captions

satisfy coursework requirements, avoid commercial cloud billing constraints, and maintain a robust, production-quality codebase. Table 8 outlines each major decision and the chosen implementation path.

Table 8: Key Architectural Decisions Log

Decision Area	Choice Made
Auth Method	Email/password only
Navigation Pattern	Sidebar drawer (not bottom nav)
Maps Provider	OpenStreetMap via flutter_map
Image Storage	Base64 strings in Firestore
Payment Gateway	Stripe sandbox (client-side)
State Management	Riverpod only
Feature Modularization	1 Dart file per feature module
Guest Check-In	Manual check-in by host
User Roles	Contextual (host or guest per event)
Free-Tier Limits	20 guests, 5 photos per event
Premium Pricing	\$9.99 per feature
Commission Model	10% platform fee; host withdraws 90%

12 Testing and Verification

12.1 Approach

Munasabat is an academic MVP and does not include an automated test suite. Verification was performed through **manual end-to-end testing** after each phase, following the exit criteria defined in the Phases document. Each phase was required to be fully demoable before the next phase began.

12.2 Phase-by-Phase Verification Checklist

Table 9 documents the manual verification performed at the conclusion of each development phase, confirming that all required exit criteria were satisfied before proceeding to subsequent milestones.

12.3 Error Handling Verification

All error states were verified to surface plain-language messages:

- **email-already-in-use** — “An account with that email already exists. Try logging in instead.”
- **wrong-password** — “Incorrect email or password. Please try again.”
- **network-request-failed** — “No internet connection. Please check your network and retry.”

- Guest list load failure — “Couldn’t load your guest list. Pull down to try again.”

12.4 Stripe Sandbox Testing

All payment flows were tested using Stripe’s official test card:

4242 4242 4242 4242 | Expiry: any future date | CVC: any 3 digits

Tested scenarios:

- **Successful gift payment** — Firestore gift document created, host wallet updated.
- **Anonymous gift** — host wallet shows “Anonymous Well-Wisher”.
- **Premium Unlimited Guests purchase** — guest cap lifted immediately.
- **Premium +100 Image Pack** — `imageLimit` incremented from 5 to 105.
- **Fund withdrawal** — available balance cleared to zero.

13 Conclusion

Munasabat successfully delivers a fully functional, cross-platform wedding planner and guest management application as an academic MVP. The application demonstrates:

- A clean, layered Flutter architecture with strict separation of concerns between UI, state, and data layers.
- Effective use of Riverpod for real-time state management driven by Firestore streams.
- A working Stripe sandbox payment integration for both guest gifting and host premium purchases, achieved without a backend server.
- A free-tier-compatible image storage strategy using Base64 encoding in Firestore.
- A polished, branded design system with consistent typography, color tokens, and component library.
- A contextual user-role model where a single account can act as both host and guest across different events.

All originally planned MVP features have been implemented and verified against the success metrics defined in the PRD.

14 AI Usage Statement

Artificial intelligence tools were used in the development of Munasabat in a limited and targeted capacity:

- **UI implementation:** AI assistance was used to help scaffold and style Flutter widget trees and generate

Table 9: Manual Verification Checklist

Phase	Exit Criteria Verified	Status
Phase 0	App builds on Android, shows correct theme and font hierarchy	Done
Phase 1	Register, logout, login, password reset email flow works	Done
Phase 2	Create event with cover image, edit metadata, delete event	Done
Phase 3	5+ checklist tasks with progress bar; expenses with accurate total	Done
Phase 4	Add, search, edit, remove guests; approve/reject join requests	Done
Phase 5	Invitation screen shows QR and Event Code; native share sheet works	Done
Phase 6	Guest joins via code, host approves, event appears in Attending tab	Done
Phase 7	Host taps Check In; guest status updates immediately in Firestore	Done
Phase 8	Guest sends test gift; host wallet reflects balance and gift record	Done
Phase 10	Host uploads photo; approved guest can view gallery with zoom viewer	Done
Phase 12	Host purchases Unlimited Guests; locked FAB becomes active immediately	Done
Phase 13	Host withdraws balance; available balance updates to zero cleanly	Done

UI boilerplate (screen layouts, form fields, card components).

- **Business logic:** All business logic, data model design, Firestore query construction, Riverpod provider architecture, Stripe integration flow, Firestore Security Rules, and architectural decisions were **implemented independently** by the developer without AI assistance.

AI tools were not used to generate the core logic of the application, the architecture documents, or this report’s technical content. Their role was limited to accelerating repetitive UI construction tasks.

References

- [1] Flutter Team. *Flutter Documentation*. Google. <https://docs.flutter.dev>
- [2] Firebase Team. *Firebase Documentation*. Google. <https://firebase.google.com/docs>
- [3] Riverpod Contributors. *Riverpod Documentation*. <https://riverpod.dev>
- [4] go_router Contributors. *go_router Package*. https://pub.dev/packages/go_router
- [5] Stripe Inc. *Stripe API Reference*. <https://stripe.com/docs/api>
- [6] flutter_stripe Contributors. *flutter_stripe Package*. https://pub.dev/packages/flutter_stripe
- [7] OpenStreetMap Foundation. *OpenStreetMap*. <https://www.openstreetmap.org>
- [8] Nominatim Contributors. *Nominatim Geocoding API*. <https://nominatim.org>
- [9] flutter_map Contributors. *flutter_map Package*. https://pub.dev/packages/flutter_map
- [10] Cloud Firestore Team. *Firestore Security Rules Reference*. Google. <https://firebase.google.com/docs/firestore/security/get-started>